

EVAGO: Comprehensive Architecture & Implementation Guide

Event-Week Travel Assistant for the General Business Conference Audience V1.0

1. EXECUTIVE SUMMARY & DESIGN MANDATE

EVAGO is an event-week travel assistant for business professionals. It aggregates conference side events, computes multi-leg transit between venues, and issues a single **EVAGO TravelPass** containing all RSVPs, tickets, QR codes, and real-time itinerary updates. This document defines the complete technical architecture for the EVAGO platform.



Core Design Principles: (1) **Event-first** — the user discovers side events before booking transit. (2) **Pass-centric** — every activated delegate receives a unified TravelPass. (3) **Event-driven processing** — delays and dispatches flow through async queues. (4) **Edge-native** — read paths stay within 50ms via Cloudflare edge nodes in Singapore, Kuala Lumpur, and Jakarta.

2. MONOREPO STRUCTURE & TECHNOLOGY STACK

TURBOREPO DIRECTORY STRUCTURE

/evago-monorepo

|

/apps

|

/web

Next.js 15 – B2C PWA & B2B Organiser Dashboard

|

/sdk

Vanilla JS event-platform embed (Vite, <18kb gzipped)

|

/pass-worker

Node.js service – PassKit signing + Google Wallet JWT

|

/packages

|

/core-db

Supabase Client, Drizzle ORM Schema, Types

|

/transit-engine

12Go/Easybook wrappers, multi-leg planner, yield logic

|

/demand-engine

Durable Object Logic, pool state, dispatch triggers

|

/pass-generator

Apple PassKit + Google Wallet pass builders

|

/event-aggregator

Eventbrite / Lu.ma sync, side-event normalizer

|

/border-intel

Checkpoint queue scrapers, delay event publishers

|

/security-utils

WebCrypto encryption, secure dispatch links

|

/workers

|

/queue-consumer

Cloudflare Queue handler (delays, dispatch, passes)

|

/border-poller

Scheduled Worker – polls border feeds every 2 min

|

pnpm-workspace.yaml

|

turbo.json

DOMAIN	TECHNOLOGY	RATIONALE & ROLE
Compute & Routing	Next.js 15 + OpenNext + Cloudflare Workers	App Router API routes compiled to V8 isolates. Zero cold starts at SG/KL/JKT PoPs. Sub-50ms read path.
Stateful Edge	Cloudflare Durable Objects	Required for demand pool state (member count, dispatch threshold). Cannot be replicated with stateless Workers or KV.
Async Processing	Cloudflare Queues	Receives delay events and dispatch triggers. Fan-out to affected itineraries without blocking request handlers.
Database	Supabase PostgreSQL 15 + Drizzle ORM	Drizzle replaces Prisma for V8 compatibility. Hyperdrive provides edge-to-Postgres connection pooling.
Realtime	Supabase Realtime	WebSocket push to delegates for delay alerts, border updates, pool fill notifications.
Caching	Cloudflare KV	Transit schedule cache (300s TTL). Write-throttled to 1/sec/key. Stale-while-revalidate via ctx.waitUntil().
Pass Worker	Node.js (Cloudflare Workers + nodejs_compat)	Isolated service for PassKit PKCS#7 signing and Google Wallet JWT generation. Separate wrangler.toml.
Payments	Stripe	Stripe Checkout for TravelPass subscriptions and single-purchase transit bookings. Solana Pay optional in Phase 3.
Analytics	PostHog + Sentry	PostHog tracks funnel (RSVP → transit click → pass purchase). Sentry captures edge runtime errors.

3. DATABASE SCHEMA — CORE ENTITIES (PART 1)

ENUMS & CORE TABLES

-- ENUMS

```
CREATE TYPE transit_mode AS ENUM (
  'BUS', 'FERRY', 'RAIL', 'PRIVATE_VAN', 'SHUTTLE', 'FLIGHT'
);
CREATE TYPE itinerary_status AS ENUM (
  'DRAFT', 'AWAITING_PAYMENT', 'CONFIRMED', 'ACTIVE',
  'COMPLETED', 'CANCELLED'
);
CREATE TYPE segment_status AS ENUM (
  'PENDING', 'CONFIRMED', 'DELAYED', 'CANCELLED', 'COMPLETED'
);
CREATE TYPE user_role AS ENUM ('DELEGATE', 'ORGANISER', 'ADMIN');
CREATE TYPE payout_status AS ENUM ('UNPAID', 'PROCESSING', 'PAID', 'FAILED');
CREATE TYPE pool_status AS ENUM ('FORMING', 'DISPATCHED', 'DEPARTED', 'CLOSED');
```

-- USERS

```
CREATE TABLE users (
  id UUID REFERENCES auth.users PRIMARY KEY,
  role user_role NOT NULL DEFAULT 'DELEGATE',
  display_name TEXT,
  origin_city TEXT, -- Default departure city for transit compute
  stripe_customer_id TEXT UNIQUE,
  secure_dispatch_key TEXT, -- Ephemeral key for encrypted route details
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- EVENTS (Main conference / anchor events)

```
CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  external_event_id TEXT UNIQUE NOT NULL, -- Eventbrite or Lu.ma ID
  organiser_id UUID REFERENCES users(id) ON DELETE SET NULL,
  name TEXT NOT NULL,
  event_week_slug TEXT, -- e.g. 'token2049-sg-2026'
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- SIDE EVENTS (Curated / aggregated within an event week)

```
CREATE TABLE side_events (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  event_id UUID REFERENCES events(id) ON DELETE CASCADE,
  external_event_id TEXT UNIQUE NOT NULL,
  organiser_id UUID REFERENCES users(id),
  title TEXT NOT NULL,
  venue_id UUID REFERENCES event_venues(id),
  starts_at TIMESTAMPTZ NOT NULL,
  capacity INT,
  listing_tier TEXT DEFAULT 'BASIC', -- BASIC, FEATURED, PREMIUM
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- EVENT VENUES

```
CREATE TABLE event_venues (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  event_id UUID REFERENCES events(id) ON DELETE CASCADE,
  label TEXT NOT NULL,
  address TEXT NOT NULL,
  geo_lat DECIMAL(9,6),
  geo_lng DECIMAL(9,6),
  city TEXT NOT NULL,
  country_code CHAR(2) NOT NULL,
  is_private BOOLEAN DEFAULT false
);
```

4. DATABASE SCHEMA — OPERATIONAL ENTITIES (PART 2)

```
ITINERARIES, SEGMENTS, DEMAND POOLS, RSVPS
-- ITINERARIES (parent entity for all journeys)
CREATE TABLE itineraries (
  id                UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id           UUID                REFERENCES users(id),
  event_id          UUID                REFERENCES events(id),
  pass_subscription_id UUID            REFERENCES pass_subscriptions(id),
  status            itinerary_status DEFAULT 'DRAFT',
  total_amount_cents INT               NOT NULL,
  currency          CHAR(3)            NOT NULL DEFAULT 'SGD',
  stripe_payment_intent_id TEXT        UNIQUE,
  stripe_event_id   TEXT               UNIQUE,
  evago_pass_url    TEXT,
  apple_pass_url    TEXT,
  google_pass_url   TEXT,
  created_at        TIMESTAMPTZ       DEFAULT NOW(),
  updated_at        TIMESTAMPTZ       DEFAULT NOW()
);

-- ITINERARY SEGMENTS
CREATE TABLE itinerary_segments (
  id                UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
  itinerary_id      UUID                REFERENCES itineraries(id) ON DELETE CASCADE,
  sequence_order    SMALLINT           NOT NULL,
  mode              transit_mode       NOT NULL,
  operator_name     TEXT               NOT NULL,
  operator_booking_ref TEXT,
  origin_name       TEXT               NOT NULL,
  destination_name  TEXT               NOT NULL,
  departure_at      TIMESTAMPTZ        NOT NULL,
  arrival_at        TIMESTAMPTZ,
  pnr_code          TEXT,
  status            segment_status     DEFAULT 'PENDING',
  amount_cents      INT                NOT NULL,
  created_at        TIMESTAMPTZ        DEFAULT NOW(),
  updated_at        TIMESTAMPTZ        DEFAULT NOW()
);

-- USER RSVPS TO SIDE EVENTS
CREATE TABLE side_event_rsvps (
  id                UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id           UUID                REFERENCES users(id),
  side_event_id     UUID                REFERENCES side_events(id),
  status            TEXT               DEFAULT 'REGISTERED',
  registered_at     TIMESTAMPTZ        DEFAULT NOW(),
  UNIQUE(user_id, side_event_id)
);

-- DEMAND POOLS (shared van / charter coordination)
CREATE TABLE demand_pools (
  id                UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
  side_event_id     UUID                REFERENCES side_events(id),
  origin_city       TEXT               NOT NULL,
  departure_window  TIMESTAMPTZ        NOT NULL,
  status            pool_status        DEFAULT 'FORMING',
  dispatch_threshold INT              DEFAULT 8,
  vehicle_type      TEXT,
  created_at        TIMESTAMPTZ        DEFAULT NOW()
);

CREATE TABLE demand_pool_members (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  pool_id           UUID REFERENCES demand_pools(id) ON DELETE CASCADE,
  user_id           UUID REFERENCES users(id),
  joined_at         TIMESTAMPTZ        DEFAULT NOW(),
  UNIQUE(pool_id, user_id)
);
```

5. DATABASE SCHEMA — FINANCIAL & BORDER TABLES (PART 3)

```
PASS PRODUCTS, SUBSCRIPTIONS, LEDGER, BORDER INTELLIGENCE, INDEXES

-- PASS PRODUCTS (multi-day event-week subscriptions)
CREATE TABLE pass_products (
  id                UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
  event_id          UUID                REFERENCES events(id),
  name              TEXT                NOT NULL,
  duration_days     INT                 NOT NULL,
  max_segments      INT                 NOT NULL,
  included_modes    transit_mode[],
  price_cents       INT                 NOT NULL,
  currency           CHAR(3)            NOT NULL DEFAULT 'SGD',
  valid_from        TIMESTAMPTZ,
  valid_until       TIMESTAMPTZ,
  stripe_price_id   TEXT                UNIQUE
);

CREATE TABLE pass_subscriptions (
  id                UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id           UUID                REFERENCES users(id),
  pass_product_id   UUID                REFERENCES pass_products(id),
  stripe_subscription_id TEXT          UNIQUE,
  status            TEXT                NOT NULL DEFAULT 'ACTIVE',
  segments_used     INT                 DEFAULT 0,
  activated_at      TIMESTAMPTZ,
  expires_at        TIMESTAMPTZ
);

-- LEDGER ENTRIES
CREATE TABLE ledger_entries (
  id                UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
  itinerary_id      UUID                REFERENCES itineraries(id),
  organiser_id      UUID                REFERENCES users(id),
  stripe_event_id   TEXT                UNIQUE NOT NULL,
  gross_amount_cents INT                 NOT NULL,
  organiser_revshare_cents INT           NOT NULL,
  platform_fee_cents INT                 NOT NULL,
  currency           CHAR(3)            NOT NULL,
  payout_status     payout_status DEFAULT 'UNPAID',
  created_at        TIMESTAMPTZ         DEFAULT NOW()
);

-- BORDER CHECKPOINTS
CREATE TABLE border_checkpoints (
  id                UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
  checkpoint_code   TEXT                UNIQUE NOT NULL,
  display_name      TEXT                NOT NULL,
  queue_minutes     INT,
  status            TEXT                DEFAULT 'NORMAL',
  last_updated_at   TIMESTAMPTZ         DEFAULT NOW()
);

-- INDEXES
CREATE INDEX idx_events_organiser ON events(organiser_id);
CREATE INDEX idx_venues_event ON event_venues(event_id);
CREATE INDEX idx_side_events_event ON side_events(event_id);
CREATE INDEX idx_rsvps_user ON side_event_rsvps(user_id);
CREATE INDEX idx_itin_user ON itineraries(user_id);
CREATE INDEX idx_itin_event ON itineraries(event_id);
CREATE INDEX idx_seg_itin ON itinerary_segments(itinerary_id, sequence_order);
CREATE INDEX idx_seg_departure ON itinerary_segments(departure_at) WHERE status = 'CONFIRMED';
CREATE INDEX idx_pool_event_window ON demand_pools(side_event_id, departure_window);
CREATE INDEX idx_ledger_organiser ON ledger_entries(organiser_id);

-- Auto-update updated_at
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_users_upd BEFORE UPDATE ON users FOR EACH ROW EXECUTE FUNCTION set_updated_at();
CREATE TRIGGER trg_itin_upd BEFORE UPDATE ON itineraries FOR EACH ROW EXECUTE FUNCTION set_updated_at();
CREATE TRIGGER trg_seg_upd BEFORE UPDATE ON itinerary_segments FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

6. ROW LEVEL SECURITY — COMPLETE POLICY SET

POSTGRESQL RLS ON ALL CORE TABLES

-- Enable RLS

```
ALTER TABLE users          ENABLE ROW LEVEL SECURITY;
ALTER TABLE events         ENABLE ROW LEVEL SECURITY;
ALTER TABLE event_venues   ENABLE ROW LEVEL SECURITY;
ALTER TABLE side_events    ENABLE ROW LEVEL SECURITY;
ALTER TABLE side_event_rsvps ENABLE ROW LEVEL SECURITY;
ALTER TABLE itineraries    ENABLE ROW LEVEL SECURITY;
ALTER TABLE itinerary_segments ENABLE ROW LEVEL SECURITY;
ALTER TABLE demand_pools   ENABLE ROW LEVEL SECURITY;
ALTER TABLE demand_pool_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE pass_subscriptions ENABLE ROW LEVEL SECURITY;
ALTER TABLE ledger_entries  ENABLE ROW LEVEL SECURITY;
```

-- USERS

```
CREATE POLICY "users_select_own" ON users FOR SELECT USING (id = auth.uid());
CREATE POLICY "users_insert_own" ON users FOR INSERT WITH CHECK (id = auth.uid());
CREATE POLICY "users_update_own" ON users FOR UPDATE USING (id = auth.uid()) WITH CHECK (id = auth.uid());
```

-- EVENTS

```
CREATE POLICY "events_public_read" ON events FOR SELECT USING (true);
CREATE POLICY "events_organiser_ins" ON events FOR INSERT WITH CHECK (organiser_id = auth.uid());
CREATE POLICY "events_organiser_upd" ON events FOR UPDATE USING (organiser_id = auth.uid()) WITH CHECK (organiser_id = auth.uid());
CREATE POLICY "events_organiser_del" ON events FOR DELETE USING (organiser_id = auth.uid());
```

-- SIDE EVENTS

```
CREATE POLICY "side_events_public_read" ON side_events FOR SELECT USING (true);
CREATE POLICY "side_events_organiser_write" ON side_events FOR ALL USING (organiser_id = auth.uid());
```

-- SIDE EVENT RSVPS

```
CREATE POLICY "rsvps_user_all" ON side_event_rsvps FOR ALL USING (user_id = auth.uid());
```

-- ITINERARIES

```
CREATE POLICY "itin_delegate_all" ON itineraries FOR ALL USING (user_id = auth.uid());
CREATE POLICY "itin_organiser_read" ON itineraries FOR SELECT USING
  (EXISTS (SELECT 1 FROM events e WHERE e.id = itineraries.event_id AND e.organiser_id = auth.uid()));
```

-- ITINERARY SEGMENTS

```
CREATE POLICY "seg_delegate_read" ON itinerary_segments FOR SELECT USING
  (EXISTS (SELECT 1 FROM itineraries i WHERE i.id = itinerary_segments.itinerary_id AND i.user_id = auth.uid()));
```

-- DEMAND POOLS

```
CREATE POLICY "pools_auth_read" ON demand_pools FOR SELECT USING (auth.uid() IS NOT NULL);
CREATE POLICY "pool_members_own" ON demand_pool_members FOR ALL USING (user_id = auth.uid());
```

-- SUBSCRIPTIONS & LEDGER

```
CREATE POLICY "sub_delegate_all" ON pass_subscriptions FOR ALL USING (user_id = auth.uid());
CREATE POLICY "ledger_organiser_read" ON ledger_entries FOR SELECT USING (organiser_id = auth.uid());
```

7. SIDE-EVENT INTEGRATION & TRAVELPASS TRIGGER MODEL

EVAGO's core entry point is **event discovery**, not transit search. The user browses side events, RSVPs, and then receives a pre-computed itinerary and TravelPass. This is the architectural inversion that separates EVAGO from generic travel aggregators.

```
Event Platform (Eventbrite / Lu.ma) —webhook→ /api/webhooks/events —enqueue→ TRANSIT_QUEUE | | | |
Verify HMAC signature Queue Consumer Worker | Extract delegate + computeTransit(origin → venues) | side_event
RSVPs | | | createDraftItinerary() | | | — EVAGO SDK embed (passive button) ← push notification ←
Supabase Realtime "Your EVAGO Travel Pass is ready"
```

APP/API/WEBHOOKS/EVENTS/ROUTE.TS – EVENT-FIRST TRIGGER

```
import { db } from '@evago/core-db';

export async function POST(req: Request, { env }: { env: Env }) {
  const sig = req.headers.get('x-event-signature') ?? '';
  const body = await req.text();

  if (!verifyEventSignature(body, sig, env.EVENT_WEBHOOK_SECRET)) {
    return new Response('Unauthorized', { status: 401 });
  }

  const payload = JSON.parse(body);

  // Trigger 1: Main conference registration confirmed
  if (payload.event_type === 'registration.confirmed') {
    const { attendee, event_id, event_slug } = payload;

    const delegate = await db.query.users.findFirst({
      where: (u, { eq }) => eq(u.email, attendee.email)
    });
    if (!delegate?.origin_city) return Response.json({ ok: true });

    await env.TRANSIT_QUEUE.send({
      type: 'compute_event_week',
      delegate_id: delegate.id,
      event_id: event_id,
      origin_city: delegate.origin_city,
      event_slug: event_slug
    });
  }

  // Trigger 2: Side-event RSVP confirmed
  if (payload.event_type === 'rsvp.confirmed') {
    await env.TRANSIT_QUEUE.send({
      type: 'compute_side_event_route',
      delegate_id: payload.attendee.id,
      side_event_id: payload.side_event.id,
      event_venue_id: payload.side_event.venue_id
    });
  }

  return Response.json({ ok: true });
}
```

Event-first architecture: The webhook handler supports both main-event registration and side-event RSVP. Each trigger computes the appropriate route segment and adds it to the delegate's draft itinerary. The user is notified only when the full TravelPass is ready — not on every single RSVP.

8. MULTI-LEG TRANSIT ENGINE

```
PACKAGES/TRANSIT-ENGINE/SRC/PLANNER.TS

import { fetch12GoRoutes, fetchEasybookRoutes } from './adapters';
import { applyYieldMarkup } from './yield';
import type { UnifiedRoute, ItineraryPlan } from './types';

export async function computeEventWeekPlan(
  origin: string,
  sideEventIds: string[],
  eventId: string,
  env: Env
): Promise<ItineraryPlan[]> {

  // Fetch all side-event venues and time windows
  const venues = await getSideEventVenues(sideEventIds, env.DB);

  // Build ordered waypoints: origin -> venue1 -> venue2 -> ...
  const waypoints = buildWaypointSequence(origin, venues);

  // Fan-out concurrent API calls across all legs
  const legResults = await Promise.allSettled(
    waypoints.map(leg => fetchLegOptions(leg, env))
  );

  const validLegs = legResults
    .filter(r => r.status === 'fulfilled')
    .map(r => (r as PromiseFulfilledResult<UnifiedRoute[]>).value);

  // Compose legs into complete event-week itinerary plans
  const plans = composeEventWeekPlans(validLegs, venues);

  // Apply operator commission yield (backend-only, no consumer markup)
  return plans.map(applyYieldMarkup).sort((a, b) => a.score - b.score);
}

async function fetchLegOptions(leg: LegQuery, env: Env): Promise<UnifiedRoute[]> {
  const cacheKey = `transit:${leg.from}:${leg.to}:${leg.date}`;
  const cached = await env.KV.get(cacheKey, 'json');
  if (cached) return cached;

  const [goRoutes, ebRoutes] = await Promise.allSettled([
    fetch12GoRoutes(leg),
    fetchEasybookRoutes(leg)
  ]);

  const merged = normalizeAndMerge(goRoutes, ebRoutes);
  ctx.waitUntil(env.KV.put(cacheKey, JSON.stringify(merged), { expirationTtl: 300 }));
  return merged;
}
```

PACKAGES/TRANSIT-ENGINE/SRC/YIELD.TS — BACKEND COMMISSION MODEL

```
interface YieldConfig {
  baseCommissionPct: number; // 3-5% operator commission
  eventWeekMultiplier: number; // 1.0-1.2x during peak demand
}

export function applyYieldMarkup(plan: ItineraryPlan): ItineraryPlan {
  const config = getYieldConfig(plan.eventWeekSlug);

  plan.segments = plan.segments.map(seg => {
    // Consumer price stays at operator parity; EVAGO earns backend commission
    seg.retail_cents = seg.wholesale_cents;
    seg.evago_commission_cents = Math.round(
      seg.wholesale_cents * config.baseCommissionPct * config.eventWeekMultiplier
    );
    return seg;
  });

  plan.total_retail_cents = plan.segments.reduce((s, seg) => s + seg.retail_cents, 0);
  plan.total_commission_cents = plan.segments.reduce((s, seg) => s + seg.evago_commission_cents, 0);
  return plan;
}
```

9. REAL-TIME ARCHITECTURE & DEMAND POOLING

Cloudflare Queues

All side-effects (event-week compute, delay cascade, dispatch triggers) are published to Cloudflare Queues and consumed asynchronously.

QUEUE	PUBLISHER	CONSUMER
TRANSIT_QUEUE	Event webhook	Compute routes, create draft itinerary, push pass-ready notification
DELAY_QUEUE	Border poller	Evaluate affected segments, cascade rebook or notify
DISPATCH_QUEUE	Durable Object threshold	Create vehicle dispatch, notify pool members
PASS_QUEUE	Payment confirmed	Generate Apple + Google Wallet passes, update itinerary

Supabase Realtime

Delegates subscribe to their personal channel on app load. Server-side events (pass ready, delay alert, pool filled) push via WebSocket without polling.

```
const ch = supabase
  .channel(`delegate:${userId}`)
  .on('broadcast', { event: 'pass_ready' },
    (payload) => showPassNotification(payload))
  .on('broadcast', { event: 'delay_alert' },
    (payload) => showDelayBanner(payload))
  .on('broadcast', { event: 'pool_dispatched' },
    (payload) => showVehicleConfirmation(payload))
  .subscribe();
```

Cloudflare Durable Objects — Demand Pool State

Each (side_event_id, departure_window) pair maps to one Durable Object instance. All pool operations (join, leave, threshold check) are serialised through the DO, avoiding race conditions on concurrent joins.

```
PACKAGES/DEMAND-ENGINE/SRC/DEMANDPOOL.TS

export class DemandPool implements DurableObject {
  private members = new Map<string, PoolMember>();
  private dispatched = false;

  constructor(private state: DurableObjectState, private env: Env) {
    this.state.blockConcurrencyWhile(async () => {
      this.members = await this.state.storage.get('members') ?? new Map();
      this.dispatched = await this.state.storage.get('dispatched') ?? false;
    });
  }

  async fetch(req: Request): Promise<Response> {
    const { action, member, threshold } = await req.json();

    if (action === 'join' && !this.dispatched) {
      this.members.set(member.userId, member);
      await this.state.storage.put('members', this.members);

      if (this.members.size >= (threshold ?? 8)) {
        await this.triggerDispatch();
      }
    }

    return Response.json({ size: this.members.size, dispatched: this.dispatched });
  }

  private async triggerDispatch() {
    this.dispatched = true;
    await this.state.storage.put('dispatched', true);
    await this.env.DISPATCH_QUEUE.send({
      memberIds: [...this.members.keys()],
      poolId: this.state.id.toString()
    });
  }
}
```

10. DIGITAL PASS GENERATION — APPLE WALLET & GOOGLE WALLET

The `/apps/pass-worker` service runs as a separate Cloudflare Worker with the `nodejs_compat` flag enabled. It handles PKCS#7 cryptographic signing for Apple and JWT signing for Google. It is triggered via the `PASS_QUEUE` on booking confirmation.

Apple Wallet (.pkpass)

```
PASS-WORKER/SRC/APPLE.TS

export async function buildApplePass(itin: Itinerary) {
  const pass = {
    formatVersion: 1,
    passTypeIdentifier: env.APPLE_PASS_TYPE_ID,
    teamIdentifier: env.APPLE_TEAM_ID,
    serialNumber: itin.id,
    organizationName: 'EVAGO',
    description: `EVAGO Travel Pass — ${itin.eventName}`,
    backgroundColor: 'rgb(255,255,255)',
    foregroundColor: 'rgb(15,23,42)',
    generic: {
      primaryFields: [{ key: 'event', label: 'EVENT WEEK', value: itin.eventName }],
      secondaryFields: itin.segments.map((s,i) => ({
        key: `seg${i}`, label: s.mode, value: s.departure_at
      })),
      barcode: { message: itin.unified_qr_payload, format: 'PKBarcodeFormatQR' }
    },
  };
  const signed = await signPassWithPKCS7(pass, env.APPLE_CERT, env.APPLE_KEY);
  return await bundleAsPkpass(signed);
}
```

Google Wallet (JWT Pass)

```
PASS-WORKER/SRC/GOOGLE.TS

import jwt from 'jsonwebtoken';

export async function buildGooglePass(itin: Itinerary): Promise<string> {
  const passObject = {
    id: `${env.GOOGLE_ISSUER_ID}.${itin.id}`,
    classId: `${env.GOOGLE_ISSUER_ID}.evago_v1`,
    genericType: 'GENERIC_TYPE_UNSPECIFIED',
    cardTitle: { defaultValue: { language: 'en', value: 'EVAGO Travel Pass' } },
    header: { defaultValue: { language: 'en', value: itin.eventName } },
    hexBackgroundColor: '#FFFFFF',
    hexForegroundColor: '#0F172A',
    barcode: { type: 'QR_CODE', value: itin.unified_qr_payload },
    textModulesData: itin.segments.map((s,i) => ({
      id: `seg_${i}`,
      header: `Leg ${i+1} — ${s.mode}`,
      body: `${s.origin_name} → ${s.destination_name}`
    })))
  };

  const claims = {
    iss: env.GOOGLE_SERVICE_ACCOUNT_EMAIL,
    aud: 'google',
    origins: ['https://evago.app'],
    typ: 'savetowallet',
    payload: { genericObjects: [passObject] }
  };
  const token = jwt.sign(claims, env.GOOGLE_SA_PRIVATE_KEY, { algorithm: 'RS256' });
  return `https://pay.google.com/gp/v/save/${token}`;
}
```

Pass Update Strategy: When a segment is delayed or rebooked, the pass-worker pushes an update. For Apple Wallet, this uses the APNs push token stored against the pass serial. For Google Wallet, the pass object is PATCHed via the Google Wallet REST API with the updated departure time and new QR payload.

11. BORDER INTELLIGENCE & CASCADE REBOOKING

CHECKPOINT	CORRIDOR	DATA SOURCE	UPDATE FREQ
Woodlands / Tuas	Singapore → Malaysia	ICA Singapore + operator status feeds	5 min
JB Sentral CIQ	Malaysia → Singapore	JMB queue + operator status feeds	5 min
Batam Ferry Terminals	Singapore → Indonesia	Batam Fast / Sindo Ferry operator feeds	10 min
KL Sentral / Puduraya	Kuala Lumpur → Singapore	Easybook + KTM status APIs	15 min

WORKERS/QUEUE-CONSUMER/SRC/DELAY-HANDLER.TS

```

async function handleDelayEvent(event: DelayEvent, env: Env) {
  const affected = await db.query.itinerary_segments.findMany({
    where: (s, { and, eq, between }) => and(
      eq(s.status, 'CONFIRMED'),
      eq(s.origin_name, event.checkpoint_name),
      between(s.departure_at, event.window_start, event.window_end)
    ),
    with: { itinerary: { with: { user: true } } }
  });

  for (const segment of affected) {
    const delayedDeparture = new Date(
      segment.departure_at.getTime() + event.delay_minutes * 60000
    );

    const nextSeg = await getNextSegment(segment.itinerary_id, segment.sequence_order);
    const missedConnection = nextSeg &&
      delayedDeparture > new Date(nextSeg.departure_at.getTime() - 20 * 60000);

    if (missedConnection) {
      const alternatives = await fetchAlternativeSegment(nextSeg, env);
      if (alternatives.length > 0) {
        await rebookSegment(nextSeg.id, alternatives[0], env.DB);
        await notifyDelegate(segment.itinerary.user, 'rebooked', alternatives[0], env);
      } else {
        await notifyDelegate(segment.itinerary.user, 'missed_connection', nextSeg, env);
      }
    } else {
      await db.update(itinerary_segments)
        .set({ status: 'DELAYED', departure_at: delayedDeparture, updated_at: new Date() })
        .where(eq(itinerary_segments.id, segment.id));
      await notifyDelegate(segment.itinerary.user, 'delayed', segment, env);
      await env.PASS_QUEUE.send({ type: 'update_pass', itinerary_id: segment.itinerary_id });
    }
  }
}

async function notifyDelegate(user: User, type: string, data: any, env: Env) {
  await env.SUPABASE_REALTIME.broadcast(`delegate:${user.id}`, { event: 'delay_alert', type, data });
  await sendEmail(user.email, buildDelayEmailTemplate(type, data));
}

```

12. PAYMENT ARCHITECTURE & DOUBLE-ENTRY LEDGER

APP/API/CHECKOUT/ROUTE.TS - IDEMPOTENT STRIPE SESSION

```
export async function POST(req: Request) {
  const { itinerary_id } = await req.json();
  const itinerary = await getItinerary(itinerary_id);

  const session = await stripe.checkout.sessions.create({
    customer:      itinerary.user.stripe_customer_id,
    mode:          'payment',
    payment_intent_data: { metadata: { itinerary_id } },
    line_items: [
      {
        quantity: 1,
        price_data: {
          currency:    itinerary.currency.toLowerCase(),
          unit_amount: 999,          -- $9.99 TravelPass fee
          product_data: { name: 'EVAGO TravelPass - Event Week Access' }
        }
      },
      ...itinerary.segments.map(seg => ({
        quantity: 1,
        price_data: {
          currency:    itinerary.currency.toLowerCase(),
          unit_amount: seg.retail_cents,
          product_data: { name: `${seg.origin_name} → ${seg.destination_name} (${seg.mode})` }
        }
      })))
    ],
    success_url: `https://evago.app/pass/${itinerary_id}?success=1`,
    cancel_url:  `https://evago.app/itinerary/${itinerary_id}`
  }, { idempotencyKey: itinerary_id });

  return Response.json({ url: session.url });
}
```

STRIPE WEBHOOK - IDEMPOTENCY VIA STRIPE_EVENT_ID UNIQUE

```
export async function handleStripeWebhook(event: Stripe.Event, db: DB) {
  if (event.type === 'payment_intent.succeeded') {
    const { itinerary_id } = event.data.object.metadata;

    await db.transaction(async (tx) => {
      await tx.update(itineraries)
        .set({ status: 'CONFIRMED', stripe_event_id: event.id })
        .where(eq(itineraries.id, itinerary_id));

      await tx.insert(ledger_entries).values({
        itinerary_id,
        organiser_id:    await getOrganiserId(itinerary_id, tx),
        stripe_event_id: event.id,          -- UNIQUE prevents replay
        gross_amount_cents: event.data.object.amount,
        organiser_revshare_cents: Math.floor(event.data.object.amount * 0.10),
        platform_fee_cents: Math.floor(event.data.object.amount * 0.90),
        currency:        event.data.object.currency.toUpperCase(),
        payout_status:    'UNPAID'
      }).onConflictDoNothing();
    });

    await env.PASS_QUEUE.send({ type: 'generate_pass', itinerary_id });
  }
}
```

Payment model: Stripe is the primary payment rail. The TravelPass fee (\$9.99) and transit segments are itemised separately on the checkout receipt for transparency. Crypto payment rails (e.g., Solana Pay) are deferred to Phase 3 and only if requested by a meaningful segment of the audience.

13. DEVOPS, CI/CD & OBSERVABILITY

```
.GITHUB/WORKFLOWS/DEPLOY.YML
name: EVAGO Production Deploy
on: { push: { branches: ["main"] } }

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
        with: { version: 9 }
      - name: Install          run: pnpm install --frozen-lockfile
      - name: Generate Drizzle  run: pnpm exec drizzle-kit generate
      - name: Lint + Typecheck  run: pnpm run lint && pnpm run typecheck
      - name: Unit Tests        run: pnpm run test

  migrate:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Apply DB Migrations
        run: pnpm exec drizzle-kit migrate
        env: { DATABASE_URL: ${{ secrets.SUPABASE_DB_URL }} }

  deploy-web:
    needs: migrate
    runs-on: ubuntu-latest
    steps:
      - name: Build          run: pnpm run build --filter=web
      - name: Deploy Next.js App
        uses: cloudflare/wrangler-action@v3
        with: { apiToken: ${{ secrets.CF_API_TOKEN }}, command: deploy }
        workingDirectory: apps/web

  deploy-pass-worker:
    needs: migrate
    runs-on: ubuntu-latest
    steps:
      - name: Build Pass Worker  run: pnpm run build --filter=pass-worker
      - name: Deploy Pass Worker
        uses: cloudflare/wrangler-action@v3
        with: { apiToken: ${{ secrets.CF_API_TOKEN }}, command: deploy }
        workingDirectory: apps/pass-worker

  deploy-queue-consumer:
    needs: migrate
    runs-on: ubuntu-latest
    steps:
      - name: Deploy Queue Worker
        uses: cloudflare/wrangler-action@v3
        with: { apiToken: ${{ secrets.CF_API_TOKEN }}, command: deploy }
        workingDirectory: workers/queue-consumer
```

14. COMPLIANCE & DATA PROTECTION

JURISDICTION	REQUIREMENT	IMPLEMENTATION
Singapore	PDPA 2012	Consent flows, data portability, breach notification within 3 days, PII scrubbing in Sentry.
Malaysia	PDPA 2010	Explicit consent for route/location data storage; encrypted route details at rest.
Indonesia	UU PDP 2022	In-ASEAN data processing; Supabase Singapore region; cross-border transfer assessment.
Thailand	PDPA 2019	Right-to-erasure workflow within 30 days; automated deletion of user PII across all tables.

15. IMPLEMENTATION TIMELINE

Phase 1 — Foundation & Core Booking (Days 1–30)

- Days 1–4: Turborepo init, Next.js 15 scaffold, Drizzle schema + migrations, Supabase provisioning (SG region).
- Days 5–8: Cloudflare Workers, Durable Objects, Queues, KV namespaces.
- Days 9–12: Supabase Magic Link + OAuth; profile with origin city.
- Days 13–18: Eventbrite / Lu.ma webhook adapter, side-event aggregation, RSVP sync.
- Days 19–22: Multi-leg transit engine with 12Go + Easybook adapters.
- Days 23–26: Stripe checkout with TravelPass + segment itemisation; ledger entries.
- Days 27–30: Queue consumer for transit compute and pass-ready notifications.

Phase 2 — Passes, Pooling & Real-Time (Days 31–60)

- Days 31–35: Pass worker — Apple PassKit + Google Wallet generation.
- Days 36–40: Demand pool Durable Objects; threshold dispatch; DISPATCH_QUEUE consumer.
- Days 41–44: Side-event organiser listing portal (Basic / Featured / Premium tiers).
- Days 45–50: Multi-day pass subscriptions; Stripe Price mapping; segment quota enforcement.
- Days 51–55: B2B organiser dashboard — live arrival tracking, RSVP analytics, revenue share.
- Days 56–60: Border poller worker; checkpoint queue feeds; delay cascade handler.

Phase 3 — Scale & Optional Crypto (Days 61–90)

- Days 61–66: Cascade rebooking polish; missed-connection manual escalation.
- Days 67–70: PDPA/UU PDP compliance; consent banner; right-to-erasure endpoint.
- Days 71–75: E2E Playwright suite — RSVP → itinerary → checkout → pass → delay → rebook.
- Days 76–80: RLS security audit; load testing at 1,000 RPS.
- Days 81–85: Optional Solana Pay / USDC integration if validated by audience demand.
- Days 86–90: Production canary deployment via Cloudflare traffic split; full rollout.

16. COST PROJECTION (10,000 ACTIVE USERS / MONTH)

SERVICE	RESOURCE	MONTHLY COST (USD)
Cloudflare	Workers Paid, KV, Queues, Durable Objects, Pages	\$15.00
Supabase	PostgreSQL Pro, Auth, Realtime, Storage, Hyperdrive	\$25.00
PassKit	Apple Wallet Pass Generation API	\$45.00
Google Wallet	Google Wallet API (JWT-based)	\$0.00
Sentry	Error tracking (Team Plan)	\$26.00
PostHog	Product analytics (free tier)	\$0.00
GitHub Actions	CI/CD for private repo	~\$10.00
Stripe	Payment processing	\$0 Fixed (~2.9% + 30¢ per tx)
Total Fixed Monthly Infrastructure Burn		~\$121.00 USD / mo